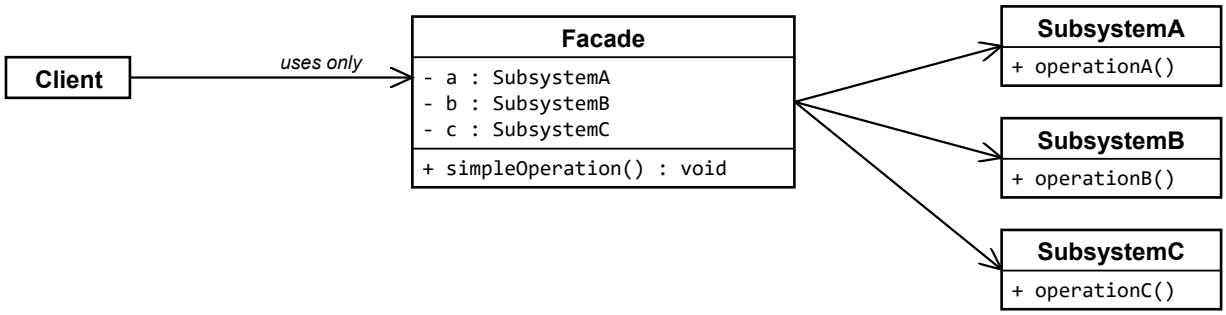
03 Facade

Structural

Intent (GoF)	Provide a **unified interface to a set of interfaces in a subsystem**. Facade defines a **higher-level interface** that makes the subsystem easier to use.
Use when	Client must call **many subsystem classes in the right order** to do one job; you want one simple entry point (“one door, many rooms”). Wedding planner, restaurant waiter, Netflix “Play”, BankService.
Trigger words	“one simple interface”, “single entry point”, “hide the complexity of”, “by internally calling inventory, payment, notification and login services”, “combine air tickets, hotels and taxis”, “complex subsystem calls scattered everywhere”.
Participants	**Facade** (knows which subsystem does what, delegates) · **Subsystem classes** (do the real work, don’t know the facade) · **Client** (talks only to the facade).
Key rules (slides)	Facade **does NOT add functionality** · it is a **wrapper / point of entry** · client **can still access subsystems directly** if needed · there can be **more than one facade** for a big system.

UML — generic

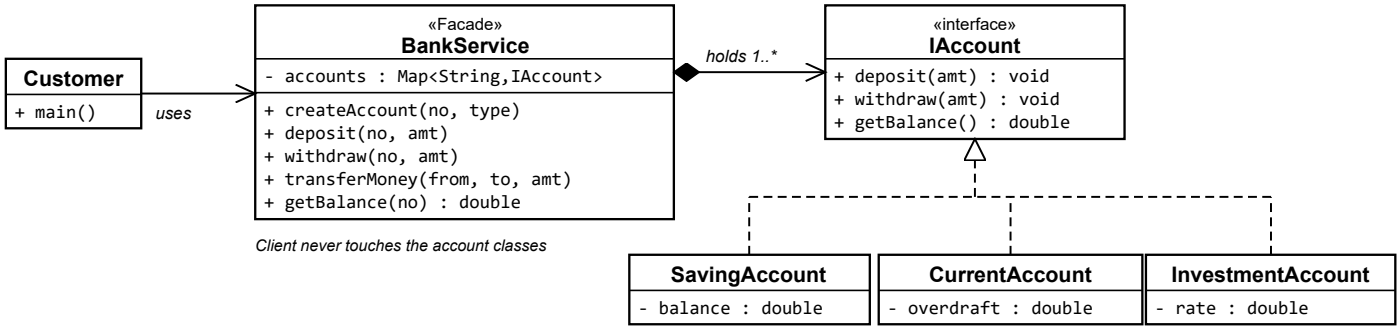


delegates; subsystems do not know the facade

Without Facade	With Facade
Client talks to Auth, Payment, Inventory, Shipping, Email…	Client talks to **one** class
High coupling, many dependencies	Low coupling
Subsystem change breaks client	Change absorbed inside facade
Hard to test / use, duplicated call sequences	Easy to use, test (mock facade), reuse

Facade vs Factory Method	
Factory	Creational · **creates** objects · “which object to create?” · `createPayment()`
Facade	Structural · **coordinates** existing objects · “what to do with many objects?” · `transferMoney()`

UML — Banking example (lecture)



Client never touches the account classes

Filled diamond = composition (facade owns the subsystem objects). Dashed + triangle = implements `IAccount`.

Java

```
public interface IAccount {
    void deposit(double amt);
    void withdraw(double amt);
    double getBalance();
}
public class SavingAccount implements IAccount {
    private double balance;
    public void deposit(double a)  { balance += a; }
    public void withdraw(double a) {
        if (a > balance) throw new
IllegalStateException("Insufficient");
        balance -= a;
    }
    public double getBalance() { return balance; }
}
// CurrentAccount, InvestmentAccount: same shape, own rules
```

```
public class BankService {                  // FACADE
    private Map<String, IAccount> accounts = new HashMap<>();

    public void createAccount(String no, String type) {
        accounts.put(no, type.equals("SAVING") ? new
SavingAccount()
                       : type.equals("CURRENT") ? new
CurrentAccount()
                       : new InvestmentAccount());
    }
    public void deposit(String no, double a)  {
accounts.get(no).deposit(a); }
    public void withdraw(String no, double a) {
accounts.get(no).withdraw(a); }
    public void transferMoney(String from, String to, double a)
{
        accounts.get(from).withdraw(a);    // coordinates a
sequence
        accounts.get(to).deposit(a);
    }
    public double getBalance(String no) { return
accounts.get(no).getBalance(); }
}
// CLIENT: one object, one entry point
BankService bank = new BankService();
bank.createAccount("John", "SAVING");
bank.createAccount("Alice", "CURRENT");
bank.deposit("John", 5000);
bank.transferMoney("John", "Alice", 1000);
```

Advantages / limitations / challenge → mechanism

Advantages	Limitations	Challenge → mechanism
Less coupling: client depends on one class	May become a **God class** (too big)	God class → split into several smaller facades per area (BookingFacade, PaymentFacade); keep it delegating only (SRP)
Easier to learn and use; cleaner code	Extra layer / abstraction	Single point of failure / bottleneck → stateless facade, run several instances behind a load balancer (HA); or a Singleton if one is enough
Easier maintenance: subsystem changes don't reach client	Not worth it for very small systems	Hides useful features → still allow direct subsystem access for advanced clients
Reusable; easy to test (mock facade)	Can hide useful subsystem functionality	Subsystems have incompatible interfaces (third-party) → put **Adapters** under the facade

Questions & model answers

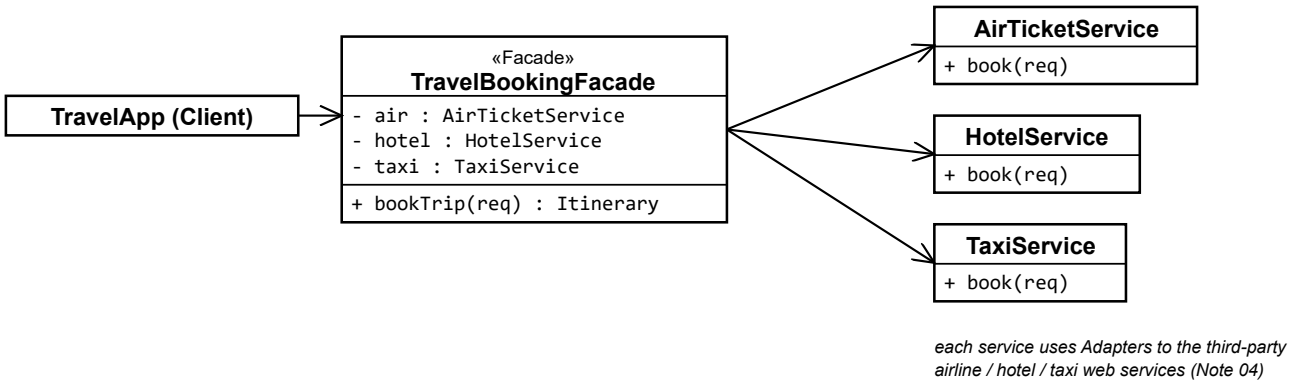
In this section: **Library borrowing** FACADE · **Travel booking** FACADE + ADAPTER · **Theory** FACADE · **Facade vs Adapter** COMPARE

FACADE **LIBRARY BORROWING** — Approach to provide one simple interface for borrowing books by internally calling inventory, payment, notification and login services. *Quiz 3 Q5*

Facade. Create `LibraryFacade.borrowBook(userId, bookId)` which, in order, calls `LoginService.verify()` → `InventoryService.checkAvailable()/reserve()` → `PaymentService.chargeFee()` → `NotificationService.sendConfirmation()`. The librarian UI calls only the facade, so it is decoupled from four services; a change in any service is handled inside the facade.

```
public class LibraryFacade {
    private LoginService login = new LoginService();      private InventoryService inv = new InventoryService();
    private PaymentService pay = new PaymentService();    private NotificationService note = new NotificationService();
    public boolean borrowBook(String user, String book) {
        if (!login.verify(user) || !inv.isAvailable(book)) return false;
        inv.reserve(book);  pay.chargeFee(user);  note.send(user, "Borrowed " + book);
        return true;
    }
}
```

FACADE + ADAPTER **TRAVEL BOOKING** — Travel website: user books air ticket, hotel and taxi; collect data from different web services. Draw a diagram; which pattern for the sub-system that combines them and why? Challenges and mechanism? *2022 Q1.1–1.2 (15)*



each service uses Adapters to the third-party airline / hotel / taxi web services (Note 04)

Facade for combining: `TravelBookingFacade.bookTrip()` calls Air, Hotel and Taxi services and returns one itinerary — user does one action, subsystem complexity hidden, low coupling. **Challenges:** (1) facade grows into a God class as services are added; (2) a failure in one service (taxi down) affects the whole booking / single point of failure; (3) partial booking consistency (air booked, hotel failed). **Mechanisms:** split into smaller facades; delegate only; timeouts + retries + fallback and run facade as stateless replicas; compensating actions (cancel air if hotel fails). For connecting each sub-service to different third-party web services → **Adapter** (Note 04).

FACADE **THEORY** — Can you access each subsystem directly? Should there be only one facade? *slide Q&A*

Yes — the facade is a convenience, not a lock; advanced clients may use subsystems directly. No — a complex system can have several facades (one per use-case area) to avoid a God class.

COMPARE **FACADE VS ADAPTER** — How is Facade different from Adapter?

Adapter **converts** one interface into another the client expects (usually wraps **one** object, makes it compatible). Facade **simplifies**: a new, higher-level interface over **many** objects. Adapter = translator, Facade = simplifier.

Mistakes

Putting business logic in the facade (it only delegates) · drawing subsystems pointing back to the facade (they don't know it) · calling it creational · confusing with Adapter (interface conversion) or Proxy (same interface, access control).